

Don't Paste In Conditioning Tokens: Noising Conditioning Tokens for Conditional Generation in Continuous Diffusion Language Models

Anonymous authors

Paper under double-blind review

Abstract

1 Fundamentally, diffusion and flow models learn a score or velocity which
 2 can be integrated via an ODE solver to map from a base distribution to
 3 data distribution. One way to parameterize their learning objective is via
 4 the x_1 prediction; i.e, predict the target data given the current noised input
 5 x_t . In practical setting for language, generation is conditional: given a
 6 prompt, generate a response. Motivated by the downstream application of
 7 conditional generation, we view the x_1 denoising objective of continuous
 8 diffusion language models as conditional prediction: given an input x_t (and
 9 optionally conditioning info c), predict the target output data x_1 . Under
 10 this view, we see the various noising levels of the training objective as
 11 curriculum learning; predict the data x_1 varying degrees of signal of the
 12 input x_t . From this perspective, we introduce a simple modification to con-
 13 tinuous diffusion language model training objective: noise the conditioning
 14 tokens as well. We demonstrate that with this simple modification, we
 15 get better generalization and generation quality, as measured by various
 16 datasets such as Sudoku, NQueens, and moreover (TODO ADD) language
 17 reasoning with GSM8k. Our method can be seen as a generalization of the
 18 classifier free guidance training set up.

19 1 Introduction

20 Fundamentally, diffusion and flow models learn a score or velocity corresponding to an
 21 ODE which can be integrated to map from a base distribution to the target data distribution.
 22 (we need to make this more precise) Their objective can be parameterized in different ways,
 23 including the denoising parametrization $\hat{x}_1(x_t)$, where the goal is to predict a clean target
 24 data x_1 given a current noisy input x_t .

25 Our framework is to view this learning objective from the lens of conditional prediction:
 26 given an noisy input, predict a clean target. We then see the varying levels of noise corrup-
 27 tion of the input as a form of curriculum learning, where the model is trained on harder
 28 (more noisier) inputs and easier (less noisier) inputs.

29 In practice for language modelling, conditional generation is the main paradigm: given a
 30 conditioning prompt, generate a response. Currently, conditional generation in continuous
 31 diffusion language models / flow language models utilize following learning objective:
 32 given conditioning tokens x^c and a noised response y_t , predict the target response y_1 . Like-
 33 wise in sampling, the conditioning tokens are fixed and non-noised. (Double check: but this
 34 is what sFLM does. Check FLM. Check ELF.) We improve this conditional learning objective
 35 by allowing conditional tokens to also be noised: given noised conditional input x_t^c , noised
 36 response y_t , predict the target response y_1 . We believe this objective induces curriculum
 37 learning, where the prediction problem $x \rightarrow y$ is made more difficult by increasing the
 38 amount of noise in the conditioning input. Moreover, (TODO: we can delete this) by noising
 39 the conditioning input, our model has a wider target that it can learn from (also needs to
 40 match the denoised conditioning input), making learning more compute efficient. Finally,

by varying the amount of conditioning signal, we believe that the learning objective is more robust and conducive to generalization. Varying the level of noise on the conditioning input can be seen as a generalization of classifier free guidance. (TODO: verify CFG). Empirically, we verify on various reasoning tasks, such as Sudoku and NQueens. We find that by additionally noising the conditioning inputs, we get better conditional generation performance on these reasoning benchmarks and we get higher levels of generation diversity.

2 Background and Related Work

Continuous diffusion and flow language models. Continuous diffusion and flow models for text learn a velocity or score that an ODE/SDE solver integrates from a base distribution to data; the denoising (x_1 -prediction) parameterization predicts the clean target from a noised input (TODO, 2025b). Conditional generation is typically implemented by holding the conditioning tokens clean throughout corruption and re-pasting them at every sampling step. Diffusion forcing (TODO, 2025e) assigns each token its own noise level, which is what lets us noise clue tokens independently of the cells we generate.

Classifier-free guidance. Classifier-free guidance (TODO, 2025a) jointly trains a conditional and an unconditional model by randomly dropping the conditioning signal. Our scheme generalizes this: rather than a binary present/absent conditioning signal, the per-token noise level continuously interpolates the strength of the conditioning, with $p = 1$ (always clean) and $p = 0$ (always noised) as the conditional-only and unconditional-only endpoints.

Corrupting conditioning / prefix tokens. Corrupting conditioning information has appeared in other settings but with different aims. Cascaded diffusion models (TODO, 2025d) corrupt the low-resolution conditioning image for robustness to upstream artifacts. In diffusion language models, Blockwise SFT (TODO, 2025c) reports that corrupting the prefix *hurts* autoregressive-style decoding—a contrast with our finding that noising clue tokens helps conditional infilling—and prompt-infilling work (TODO, 2025g) fine-tunes masked diffusion models for infilling without noising the conditioning. Unlike the image cascade setting, our inference-time conditioning input is always clean; the noising acts purely as a training-time regularizer/curriculum.

Combinatorial reasoning benchmarks. We evaluate on Sudoku (TODO, 2025h) and N-Queens (TODO, 2025f), constraint-satisfaction tasks where solving unseen instances requires learning the rules rather than memorizing input/output pairs; N-Queens additionally admits many solutions per board, enabling a coverage (diversity) metric.

3 Method

[Take 1] Our method is simple and requires minimal modification from the default training objective of flow language models. It can also be seen as a generalization of classifier free guidance. Simply put, with some probability p_{clean} , we keep the conditioning tokens clean; else, we noise the conditioning tokens as well.

During sampling, following the diffusion/flow language model literature, we keep the conditioning tokens clean and paste clean tokens at each sampling step. To prevent a large training and sampling mismatch, with some probability p_{clean} we choose to keep the conditioning tokens clean to supply as input. The model then predicts using inputs $(x_{t=1}, y_t)$, where $x_{t=1}$ is the clean conditioning tokens and y_t is the noised response tokens.

At the same time, we do not want the model to become too reliant on the conditioning tokens and learn a brittle mapping $x \rightarrow y$. Thus, with the remaining probability $1 - p_{clean}$, we noise our conditioning tokens, reducing the amount of conditioning signal the model has access to. It receives as input both noised conditioning tokens and noised response tokens (x_t, y_t) . As visualized in Figure 1, (TODO: add Fig link) by additionally noising our conditioning tokens, our model is trained under a broader curriculum of conditional prediction tasks,

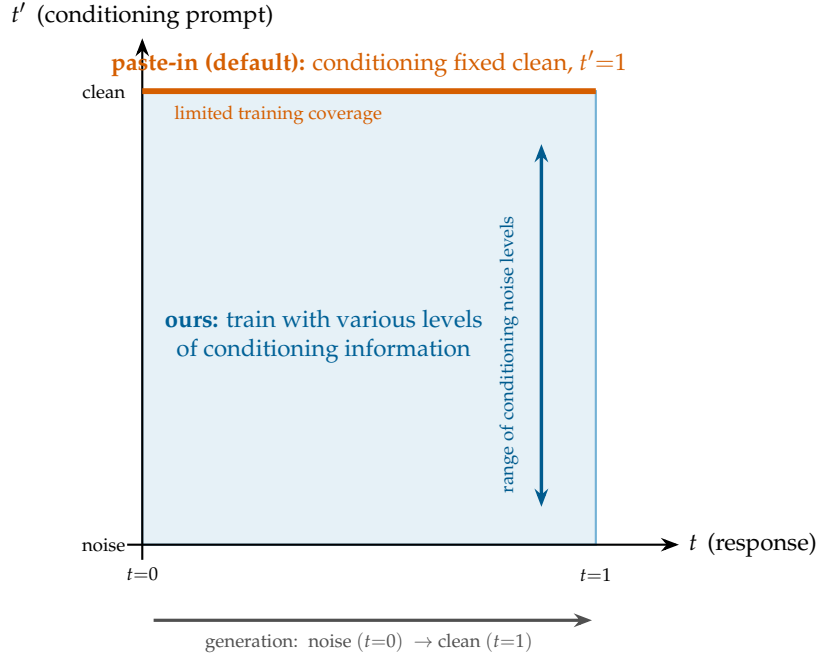


Figure 1: Our method. Standard conditional training pastes in clean conditioning tokens, covering only the top edge ($t'=1$) of the training space (orange). Instead, we train over a range of conditioning noise levels (blue region), exposing the model to a curriculum of conditioning strengths. At generation time we still integrate the response from noise ($t=0$) to clean ($t=1$) with the conditioning held clean.

90 where the model receives both a lot of conditioning signal (making prediction task easier)
 91 and little conditioning signal (making prediction task harder).

92 By default we also make use of the conditional tokens in the loss and have the model predict
 93 denoised versions of the conditioning tokens x_t . This utilizes all the tokens in the denoising
 94 objective, making the training objective more compute efficient. We discuss the effect of this
 95 in (TODO: add ablation section)

96 We can see our method as a generalization of CFG: we see full noise $t = 0$ as analogous to
 97 dropping the conditioning information $x_t = \emptyset$, and we see no noise $t = 1$ as preserving the
 98 conditioning tokens.

99 [Take 2] TODO: Can add if it's better.

100 3.1 Setup

101 We work in an *in-place infill* layout. Each example is a single board of length L (Sudoku:
 102 $L = 81$; N-Queens: $L = n^2$) over a small vocabulary. A binary *conditioning mask* marks the
 103 clue cells $\mathcal{C}$ that are revealed to the model; the complementary generation cells $\mathcal{G}$ are to be
 104 predicted. We compute the loss over the full board (clue and generation cells alike); only
 105 the clue cells are eligible to be held clean (Section 3.2).

106 3.2 Corruption process and the conditioning knob

107 We adopt a continuous-time flow / x_1 -prediction model with per-token noise levels (*diffusion*
 108 *forcing* (TODO, 2025e)): each position i carries its own time $\tau_i \in [0, 1]$, where $\tau_i = 1$ denotes
 109 the clean token and $\tau_i = 0$ full noise, and $x_i(\tau_i)$ is the corresponding corrupted input. The
 110 denoiser f_θ receives the noised board together with the per-token times and predicts the
 111 clean target $\hat{x}_1$.

A single scalar $p \equiv p_{\text{condclean}}$ controls how the clue tokens are corrupted. For each example we draw a shared time $t \sim \mathcal{U}(0, 1)$ and set

$$\tau_i = t \quad (i \in \mathcal{G}), \quad \tau_i = \begin{cases} 1 & \text{w.p. } p \quad (\text{clue held clean}), \\ t & \text{w.p. } 1 - p \quad (\text{clue noised to the board level}), \end{cases} \quad (i \in \mathcal{C}).$$

The training objective is the cross-entropy between the prediction and the clean board over the loss region $\mathcal{L}$ (the full board by default):

$$\mathcal{L}(\theta) = \mathbb{E}_{x_1, t, \{\tau_i\}} \left[\sum_{i \in \mathcal{L}} \text{CE}(f_\theta(x(\tau))_i, (x_1)_i) \right].$$

Setting $p = 1$ recovers the standard “paste-in” conditional objective (clues always clean); $p = 0$ recovers a fully unconditional objective (clues noised exactly like the rest of the board). Intermediate p interpolates between the two, exposing the model to a curriculum of conditioning strengths within a single training run.

3.3 Sampling

At inference we follow standard practice for conditional flow language models: the conditioning input is always clean, so clue positions are clamped to their clean one-hot values and re-pasted after every solver step, *independent of the training p* . We integrate the generation cells with a first-order Euler ancestral sampler. Because training with $p < 1$ exposes the model to clean clues only a p fraction of the time, a small amount of clean exposure is still required to avoid a train/inference mismatch at the always-clean test condition; we return to this in Section 5.

3.4 Metrics

We report two metrics. **Accuracy** is the fraction of generated boards that are valid solutions to the puzzle (pooled over all puzzles and samples). **Coverage** (N-Queens) is, per puzzle, the fraction of *distinct* valid solutions recovered across the k samples, averaged over puzzles; it measures the diversity of the generative distribution rather than single-shot correctness. We report 95% confidence intervals obtained by bootstrap resampling over the evaluation puzzles (10,000 resamples; for N-Queens the resampling is over the 750 input boards, with per-board accuracy and coverage computed from that board’s 20 generations).

4 Experiments

4.1 Setup

Across all experiments the backbone is a DiT-style transformer (DDiT) with hidden size 512, 8 blocks, 8 attention heads, conditioning dimension 128, and dropout 0.1, trained with a flow objective and x_1 -prediction under a log-linear noise schedule. We optimize with Adam¹ (learning rate 3×10^{-4} , $(\beta_1, \beta_2) = (0.9, 0.999)$, no weight decay), a constant schedule with 2,500 warmup steps, gradient clipping at 1.0, bf16-mixed precision, and an EMA of 0.9999. We use antithetic time sampling and a t -curriculum that anneals the sampled time range from $[0, 0]$ to $[0, 1]$ over 100k steps. Unless noted, we sample with a 128-step Euler ancestral solver at temperature 1.0. The single swept variable is $p_{\text{condclean}} \in \{0.2, 0.5, 1.0\}$ (and 0.0 where noted); all other settings are held fixed. Full hyperparameters are in Appendix A. The training GPU model is [TODO], and whether the vocabulary includes additional special tokens is [TODO].

4.2 Sudoku (Easy)

Sudoku is a (add short introduction, can borrow from SFLM or other papers). We believe that Sudoku is a good reasoning benchmark because of its combinatorial nature: in order

¹The optimizer class (Adam vs. AdamW) is [TODO] to confirm against the original configuration.

Table 1: Sudoku (easy) infill accuracy (%) on 2,000 held-out puzzles vs. the conditioning clean probability $p_{\text{condclean}}$, with 95% bootstrap CIs (10,000 resamples) in brackets. Lower p (more heavily noised clues) yields higher accuracy; the ordering is preserved across data scale. $p=1.0$ is the standard “paste-in” setup; $p=0.0$ is fully unconditional.

$p_{\text{condclean}}$	$N = 10,000$	$N = 1,000$
0.0 (unconditional)	2.85 [2.15, 3.60]	0.50 [0.20, 0.85]
0.2	86.60 [85.10, 88.10]	11.00 [9.65, 12.45]
0.5	74.75 [72.85, 76.65]	9.90 [8.60, 11.25]
1.0 (paste-in)	56.05 [53.90, 58.20]	2.30 [1.65, 2.95]

to solve unseen puzzles, the model simply learn a mapping from $x \rightarrow y$, it must implicitly learn the constraints of the game.

We train on the *easy* split in the in-place infill layout: a full 9×9 board (length 81), with the loss taken over all board tokens. We sweep $p_{\text{condclean}}$ at two training-set sizes, $N = 10,000$ and $N = 1,000$ puzzles, training for 160k steps at batch size 32 and evaluating accuracy on 2,000 held-out puzzles (Table 1).

Accuracy decreases monotonically as $p_{\text{condclean}}$ increases from 0.2 to 1.0: at $N = 10,000$ the heavily-noised setting reaches 86.6% versus 56.05% for the standard always-clean clues, a 30.6-point gap, and the same ordering holds at $N = 1,000$ despite the much lower absolute accuracy. The fully-unconditional endpoint ($p = 0.0$) collapses (2.85% / 0.5%): noising the clues helps, but the model still needs to see clean clues some of the time to match the always-clean inference condition. The best setting is thus an intermediate-but-low p , not the extreme. The 95% bootstrap CIs are non-overlapping across p at $N = 10,000$, so those separations are significant; at $N = 1,000$ the $p=0.2$ and $p=0.5$ intervals overlap (that gap is not significant), though both clearly exceed $p=1.0$.

4.3 N-Queens

NQueens is a (add short introduction, can borrow from GRAM.) In additional to being combinatorial in nature, in NQueens there can be multiple solutions for a given starting board. This allows us to measure the diversity of the generated solutions. Following GRAM, we track coverage of possible solutions and report the number of unique solutions generated out of the total number of possible solutions corresponding to that board. We evaluate on held out initial board states and generate 20 samples per board state. (TODO: add more experimental details)

We use the infill layout on an $n \times n$ board (length n^2) with token vocabulary $\{0=\text{pad}, 1=\text{empty}, 2=\text{queen}\}$, sweeping $p_{\text{condclean}}$ at two scales: $n = 8$ (length 64) and $n = 10$ (length 100). Training and evaluation splits are disjoint by clue board. For $n = 8$ this yields 1,758 training / 174 validation puzzles; the $n = 10$ train/validation counts are [TODO]. We train at global batch size 256 and evaluate the *last* checkpoint on 750 held-out puzzles with 20 generations each (Table 2).

The discriminating effect is in coverage. At $n = 8$ accuracy is high throughout (97.7–99.2%, a ~ 1.6 -point spread) while coverage spans ~ 9 points (97.31 $\rightarrow$ 88.07%) and is monotonic in p . At $n = 10$, $p = 0.2$ dominates decisively on both metrics (a ~ 30 -point accuracy lead and a ~ 9 -point coverage lead over the next best). Coverage remains monotonic (80.01 $\rightarrow$ 71.05 $\rightarrow$ 64.03%), but accuracy is *not*: $p = 0.5$ (57.42%) falls below $p = 1.0$ (64.56%). The robust cross-task claim is therefore the coverage ordering and the dominance of heavily-noised conditioning ($p = 0.2$), rather than strict accuracy monotonicity. The 95% bootstrap CIs for the $p=0.5$ ([55.88, 58.95]) and $p=1.0$ ([62.90, 66.21]) accuracies at $n = 10$ do not overlap, so the inversion is not an artifact of evaluation sampling; it is a single training

Table 2: N-Queens infill accuracy and coverage (%), last checkpoint, 750 puzzles $\times$ 20 samples, with 95% bootstrap CIs (10,000 resamples over input boards) in brackets. Coverage is monotonic in $p_{\text{condclean}}$ at both scales and $p=0.2$ dominates throughout. Note the accuracy non-monotonicity at $n = 10$ ($p=0.5 < p=1.0$); see text.

$p_{\text{condclean}}$	$n = 8$ (length 64)		$n = 10$ (length 100)	
	Accuracy	Coverage	Accuracy	Coverage
0.2	99.24	97.31	94.66	80.01
	[98.86, 99.55]	[96.52, 98.05]	[94.05, 95.23]	[78.73, 81.35]
0.5	98.75	91.41	57.42	71.05
	[98.20, 99.23]	[90.02, 92.74]	[55.88, 58.95]	[69.31, 72.77]
1.0 (paste-in)	97.67	88.07	64.56	64.03
	[96.73, 98.51]	[86.46, 89.63]	[62.90, 66.21]	[62.37, 65.70]

run, however, so training-seed variance remains unchecked and a multi-seed confirmation is left to future work.

5 Analysis and Discussion

Why does noising the conditioning help? We offer three (non-exclusive) readings, consistent with the evidence above. First, noising the clues acts as a *regularizer* against a brittle copy-and-lookup solution: when clues are always clean, the model can rely on a shortcut $x \rightarrow y$ mapping, whereas forcing it to sometimes infer the answer from noised clues pushes it to internalize the task constraints. Second, this manifests as *mode coverage*: always-clean training ($p = 1$) collapses toward fewer solutions, which is exactly what the monotonic coverage drop on N-Queens shows (Table 2). Third, the sweep can be read as a *curriculum*: a single $p < 1$ run mixes easy (clean-clue) and hard (noised-clue) conditional prediction problems within training.

As a probe of the first reading, we masked all attention into the non-conditioning tokens at inference; a model that had merely learned to copy clue $\rightarrow$ solution would be largely unaffected, but solve rate dropped to 0%, indicating the model genuinely uses the generated cells rather than a lookup table.

Relation to classifier-free guidance. Because p interpolates between a conditional-only ($p = 1$) and unconditional-only ($p = 0$) objective, our scheme can be viewed as a continuous generalization of classifier-free guidance training. We do not yet apply guidance at sampling time; combining the two is natural future work.

Limitations. Our models are small (8-block DDiT) and trained on small datasets, with heavy epoch repetition (e.g. $\sim 1.8\text{k}$ N-Queens-8 examples over 100k steps at batch 256), so overfitting/data-efficiency effects are entangled with the conditioning-noise effect. Our bootstrap CIs control evaluation-sampling variance, but all runs are single training seeds, so training-seed variance is unaddressed. Several configuration details remain to be confirmed (optimizer class, GPU, N-Queens-10 split sizes). We study easy Sudoku and small N-Queens; harder Sudoku and natural-language reasoning (GSM8k) are in progress and not reported here.

6 Conclusion

We revisited a default of conditional continuous diffusion language models—pasting in clean conditioning tokens at every step—and showed that simply *noising* the conditioning tokens during training, controlled by a single probability $p_{\text{condclean}}$, consistently improves conditional generation. Lower p (more noising) raises Sudoku infill accuracy and, most clearly, N-Queens solution coverage, with an intermediate-but-low $p \approx 0.2$ best across

tasks. The method is a one-line change to the training corruption and can be seen as a continuous generalization of classifier-free guidance. Future work includes the random-conditioning-time variant, larger models and harder Sudoku, sampling-time guidance, multi-seed variance checks, and language reasoning (GSM8k).

References

- TODO. [todo] classifier-free diffusion guidance, 2025a. Placeholder; supply arXiv link.
- TODO. [todo] flow language model, 2025b. Placeholder; supply arXiv link.
- TODO. [todo] blockwise sft for diffusion language models, 2025c. Placeholder; supply arXiv link.
- TODO. [todo] cascaded diffusion models for high fidelity image generation, 2025d. Placeholder; supply arXiv link.
- TODO. [todo] diffusion forcing, 2025e. Placeholder; supply arXiv link.
- TODO. [todo] gram / n-queens coverage source, 2025f. Placeholder; supply arXiv link.
- TODO. [todo] unlocking prompt infilling capability for diffusion language models, 2025g. Placeholder; supply arXiv link.
- TODO. [todo] sudoku-extreme / trm benchmark source, 2025h. Placeholder; supply arXiv link.

A Full Hyperparameters

Table 3 lists the shared training configuration (N-Queens Run; Sudoku differs only in board length 81, batch size 32, and 160k training steps).

Table 3: Shared hyperparameters. [TODO] items remain to be confirmed against the original configuration.

Group	Setting	Value
Model	hidden size / blocks / heads	512 / 8 / 8
	cond. dim / dropout	128 / 0.1
	board length	81 (Sudoku), 64/100 (N-Queens $n=8/10$)
Algorithm	parameterization / pred. type	mean / x_1
	loss type	flow
	diffusion forcing	true
	$p_{\text{condclean}}$	{0.2, 0.5, 1.0} (swept)
Noise	type / eps	log-linear / 0
Training	max steps	100,001 (N-Queens), 160k (Sudoku)
	global batch size	256 (N-Queens), 32 (Sudoku)
	precision / EMA	bf16-mixed / 0.9999
	grad clip	1.0
	t -curriculum	0.0 $\rightarrow$ 1.0 over 100k steps
Optimizer	class	Adam/AdamW [TODO]
	lr / (β_1, β_2) / wd	3×10^{-4} / (0.9, 0.999) / 0
	warmup	2,500 steps (constant after)
Sampling	predictor / solver	ancestral / Euler
	steps / temperature / nucleus	128 / 1.0 / 1.0
Hardware	accelerator / GPU	cuda, 1 device / [TODO]